

Technical Term Paper: Comparative Analysis and Implementation of Seq2Seq Architectures

Format: Technical Report & Source Implementation

Weight: 30 mks

Case Study: The Google Show and Tell Framework

Google's landmark work in automated image captioning serves as our reference. They replaced the standard fixed-vector encoder approach—which forces the entire spatial meaning of an image into a single point—with a dynamic decoder that can attend to specific pixels while generating each word. You are expected to replicate the logic behind this evolution.

Assignment Requirements

a) **Critical Analysis (3 Points)**

Provide a technical discussion (1 page) on why the "Simple RNN" architectures of the early 90s are fundamentally unsuitable for captioning long sequences. Referencing Lecture 10, explain exactly how the **Forget Gate** in an LSTM or the **Update Gate** in a GRU prevents the geometric decay of gradients.

Note: Focus on the mathematical necessity of the additive identity in the long-term memory cell.

b) **Feature Map Extraction (4 Points)**

Develop a Python function to load a pre-trained CNN. Most people just pull the last layer (the 1000-class vector); for this task, you must modify the model to return the **convolutional feature maps**. Comment on why a 7 x 7 or 14 x 14 grid is more useful for Attention than a single 1 x 1024 vector.

c) **Gated Decoder Implementation (4 Points)**

Code a custom recurrent cell class (LSTM or GRU). The forward method must handle the embedding of the previous word and the previous hidden state. Ensure your initialization handles the cold start problem—how do you pass the image features into the RNN for the very first timestep?

d) **Luong Multiplicative Attention Layer (5 Points)**

Build the Attention class using the Multiplicative scoring method. You must implement the dot product between the target hidden state (h_t) and the source hidden states (h_s). If the dimensions don't match, you must implement the general score method using a weight matrix W_a to bridge the two.

e) **Context & Hidden State Merging (3 Points)**

Write the code to calculate the Softmax weights and generate the final context vector. Show the concatenation of this vector with the decoder's hidden state. This is the attentional hidden state that will eventually decide the next word.

f) **The Tanh Mapping (2 Points)**

According to the sequence-to-sequence summary in Lecture 14, there is a specific transformation step before the final Softmax. Implement this fully connected layer using a **Tanh** activation. Explain in a brief comment why Tanh is preferred here over ReLU.

g) **Pipeline Development (5 Points)**

Develop the training loop. You are required to feed the ground-truth token from the dataset back into the decoder at each step. Implement Cross-Entropy Loss and explain how you handle the <padding> tokens so they don't skew your loss calculation.

h) **Inference & Behavioral Observation (4 Points)**

Implement a Greedy Search inference function (no ground truth allowed here). Use your model to describe a sample image. To conclude your paper, provide a short paragraph describing where the model's focus (attention weights) shifted when generating a noun versus a verb.